Deep Network Development

Autumn 2026, Practice #6

Viktor Varga
Department of Artificial Intelligence, ELTE IK

NumPy - View arrays

View arrays share the underlying memory with other array(s)

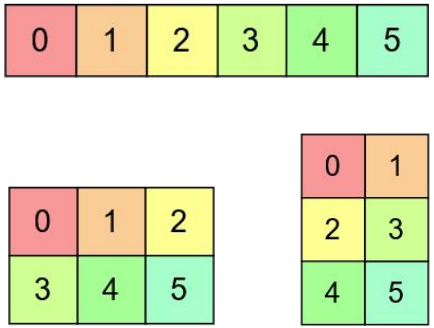
```
a6 = np.arange(6, dtype=np.int64)          # a6.shape == (6,)

a2x3 = a6.reshape((2, 3))                  # a2x3.shape == (2, 3)
a3x2 = a2x3.reshape((3, 2))                # a3x2.shape == (3, 2)

a3x2[::2, 1] = 99

print(a3x2)                                #    [[0 99]
                                           #     [2  3]
                                           #     [4 99]]

print(a6)                                  #    [0 99 2 3 4 99]
```



NumPy - View arrays

View arrays share the underlying memory with other array(s)

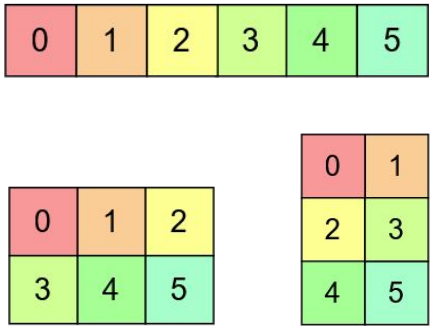
```
a6 = np.arange(6, dtype=np.int64)        # a6.shape == (6,)

a2x3 = a6.reshape((2, 3))                # a2x3.shape == (2, 3)
a3x2 = a2x3.reshape((3, 2))              # a3x2.shape == (3, 2)

a3x2[::2, 1] = 99

print(a3x2)                              #    [[0 99]
                                         #     [2  3]
                                         #     [4 99]]

print(a6)                                #    [0 99 2 3 4 99]
```



Modifying any of the views or the base array will affect all of them.

NumPy - Operations which create views

- **Reshaping**
- **Swapping axes** (e.g., transpose)
- **Basic indexing**
- **Broadcasting**
- **…**

But how do views work?

NumPy - Strides

Monthly temperatures for 24 months.

```
temperatures = np.array([-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6,
-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4], dtype=np.float32)
```

- `temperatures.size` **24**
- `temperatures.ndim` **1**
- `temperatures.shape` **(24,)**
- `temperatures.dtype` **float32**
- `temperatures.itemsize` **4**
- `temperatures.strides`
- `temperatures.base`

NumPy - Strides

Monthly temperatures for 24 months.

```
temperatures = np.array([-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6,
-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4], dtype=np.float32)
```

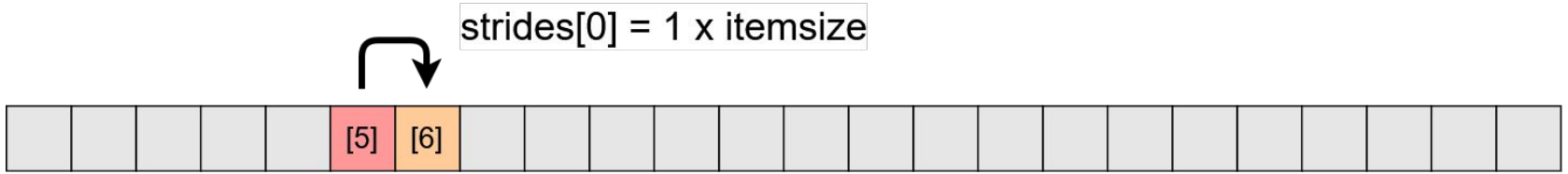
- `temperatures.size` `24`
- `temperatures.ndim` `1`
- `temperatures.shape` `(24,)`
- `temperatures.dtype` `float32`
- `temperatures.itemsize` `4`
- **`temperatures.strides`** **`(4,)`**
- **`temperatures.base`** **`None`**

NumPy - Strides

Monthly temperatures for 24 months.

```
temperatures = np.array([-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6,
-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4], dtype=np.float32)
```

- `temperatures.shape` `(24,)`
- `temperatures.dtype` `float32`
- `temperatures.itemsize` `4`
- **`temperatures.strides`** **`(4,)`**



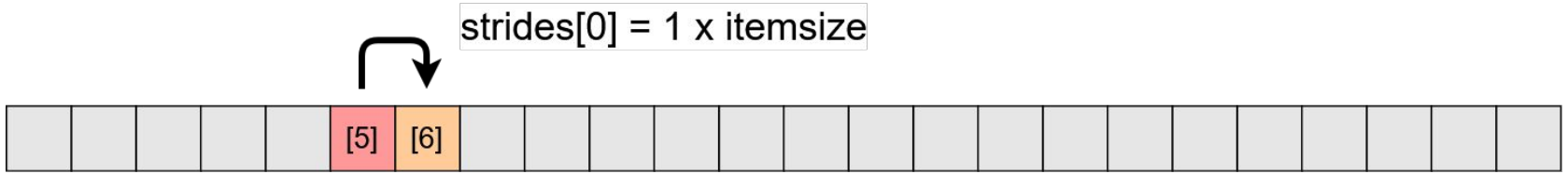
NumPy - Strides

`ndarray.strides`: A tuple of length **`ndarray.ndim`**.
If we want to jump from index **`i`** to **`i+1`** along an axis,
how many bytes do we need to jump in memory?

Monthly temperatures for 24 months.

```
temperatures = np.array([-0.5, -1.5, 5.9, 12.9, 18.1, 21.6, 23.1, 24.2, 15.0, 11.7, 3.7, -3.6,
-4.3, -3.5, 3.1, 13.5, 17.4, 25.1, 28.3, 24.3, 18.5, 13.1, 0.3, -4.4], dtype=np.float32)
```

- `temperatures.shape` `(24,)`
- `temperatures.dtype` `float32`
- `temperatures.itemsize` `4`
- **`temperatures.strides`** **`(4,)`**



NumPy - Operations which create views

- **Reshaping**
- **Swapping axes** (e.g., transpose)
- **Basic indexing**
- **Broadcasting**
- …

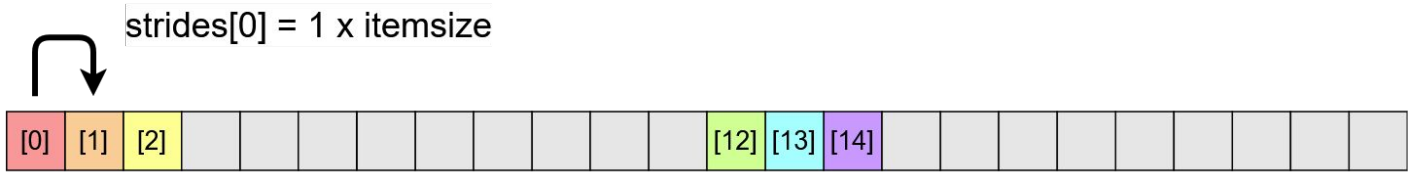
NumPy - Strides & Reshape

View the array as a 2 year long time series, each year with 12 months.

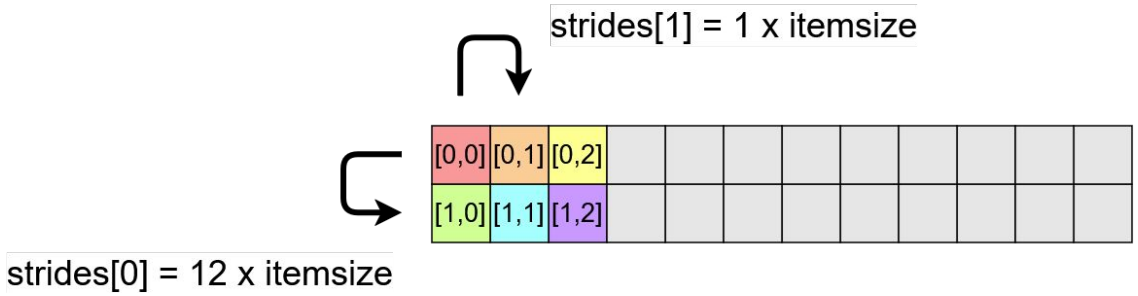
```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
print(temps2d.base)                       # <the `temperatures` array>
print(temps2d.base is temperatures)       # True
```

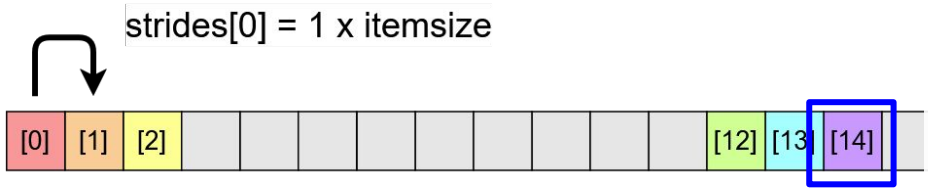
- `temperatures.shape` `(24,)`
- `temperatures.itemsize` `4`
- **`temperatures.strides`** **`(4,)`**



- `temps2d.shape` `(2, 12)`
- `temps2d.itemsize` `4`
- **`temps2d.strides`** **`(48, 4)`**

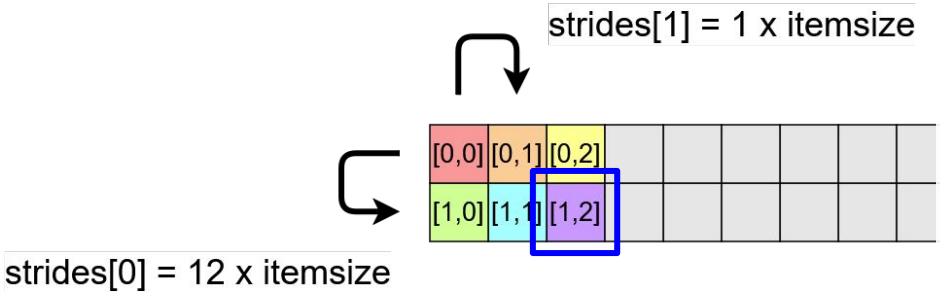


- `temperatures.shape` `(24,)`
- `temperatures.itemsize` `4`
- **`temperatures.strides`** **`(4,)`**



Example: Accessing `temperatures[14]` → Memory address of `temperatures[0]` + `14 × temperatures.strides[0]` = `temperatures[0] + 14 * 4 bytes` = `temperatures[0] + 56 bytes`

- `temps2d.shape` `(2, 12)`
- `temps2d.itemsize` `4`
- **`temps2d.strides`** **`(48, 4)`**



Example: Accessing `temps2d[1, 2]` → Memory address of `temps2d[0, 0]` + `1 × temps2d.strides[0]` + `2 × temps2d.strides[1]` = `temps2d[0, 0] + 1 * 48 + 2 * 4 bytes` = `temps2d[0, 0] + 56 bytes`

NumPy - Strides & Reshape

View the array as a 2 year long time series, each year with 12 months.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
print(temps2d.base)                       # <the `temperatures` array>
print(temps2d.base is temperatures)       # True

temps2d[[0,0,1], [5,7,7]] = 99
print(temps2d)                            # [[-0.5 -1.5  5.9 12.9 18.1 99.  23.1 99.  15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 99.  18.5 13.1  0.3 -4.4]]
print(temperatures)                       # [-0.5 -1.5  5.9 12.9 18.1 99.  23.1 99.  15.  11.7  ...   -4.4]
```

Upon modifying the view, the base array is affected too...

NumPy - Strides & Reshape

View the array as a 2 year long time series, each year with 12 months.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
print(temps2d.base)                       # <the `temperatures` array>
print(temps2d.base is temperatures)       # True
```

`ndarray.base`: A reference to the base array of the view.

NumPy - Strides & Reshape

View the array as a 2 year long time series, each year with 12 months.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
print(temps2d.base)                       # <the `temperatures` array>
print(temps2d.base is temperatures)       # True
```

`ndarray.base`: A **reference to the base array** of the view.

- A **base array** is such an array for which **a new storage area was allocated** in memory.
- **View arrays are derived from base arrays** and provide access to (some of) the elements of the base array.

NumPy - Strides & Reshape

View the array as a 2 year long time series, each year with 12 months.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
print(temps2d.base)                       # <the `temperatures` array>
print(temps2d.base is temperatures)       # True

print(temperatures.base)                  # None
```

`ndarray.base`: A **reference to the base array** of the view.

- If the array is a base array, **`ndarray.base is None`**

NumPy - Strides

- `temps2d.shape` `(2, 12)`
- `temps2d.strides` `(48, 4)`

An ndarray object is simply a re-interpretation of the underlying memory.

NumPy - Strides

- `temps2d.shape` **`(2, 12)`**
- `temps2d.strides` **`(48, 4)`**

An ndarray object is simply a re-interpretation of the underlying memory. It stores:

- The **address of the first element** (`[0], [0,0], …`) in memory.
  (e.g., `ndarray.ctypes.data`)
- **How to access its elements in the memory?**
  - **Until what index can we iterate** along an axis? (**`ndarray.shape`**)
  - **To increase an index by one along an axis,**
    how many bytes do we have to jump in memory? (**`ndarray.strides`**)

NumPy - Strides

When a new view is created, **these have to be defined only.**
No need to copy values from the underlying memory.
→ **Very efficient.**

An ndarray object is simply a re-interpretation of the underlying memory.
It stores:

- The **address of the first element** (`[0]`, `[0,0]`, …) in memory.
  (e.g., `ndarray.ctypes.data`)
- **How to access its elements in the memory?**
  - **Until what index can we iterate** along an axis? (**`ndarray.shape`**)
  - **To increase an index by one along an axis,**
    how many bytes do we have to jump in memory? (**`ndarray.strides`**)

NumPy - Operations which create views

- **Reshaping**
- **Swapping axes** (e.g., transpose)
- **Basic indexing**
- **Broadcasting**
- **…**

NumPy - Strides & Swapping axes

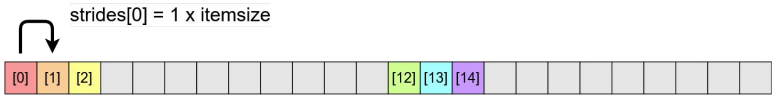
Transpose the array: Let the months axis be the first.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
```

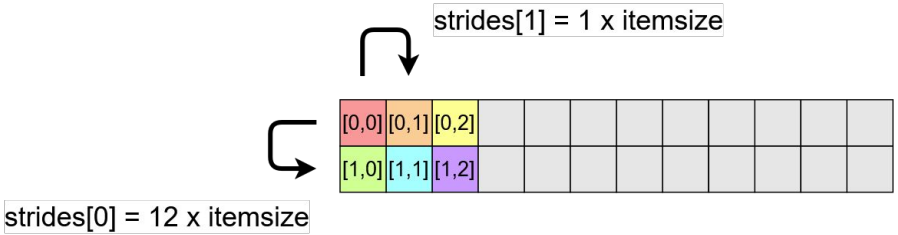
```
temps2d_T = temps2d.T

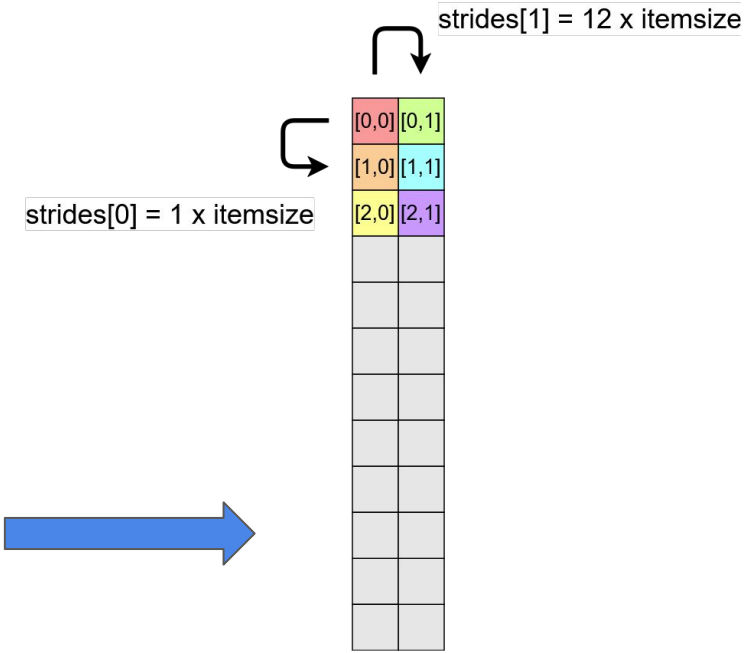
print(temps2d_T.shape)                    # (12, 2)
print(temps2d_T.strides)                  # (4, 48)
print(temps2d_T.base is temps2d)          # False
print(temps2d_T.base is temperatures)     # True
```



NumPy - Strides & Swapping axes

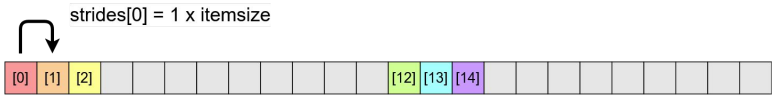
- `temps2d.shape` `(2, 12)`
- `temps2d.strides` `(48, 4)`





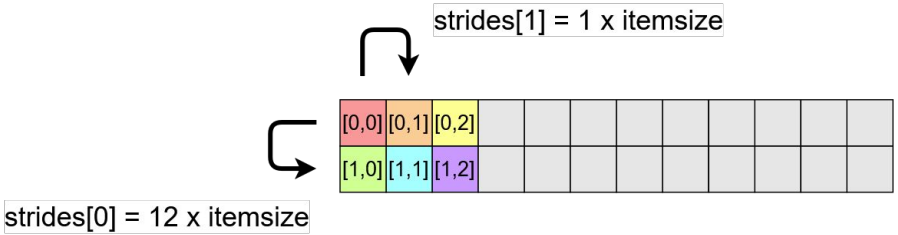

```
temps2d_T = temps2d.T
```

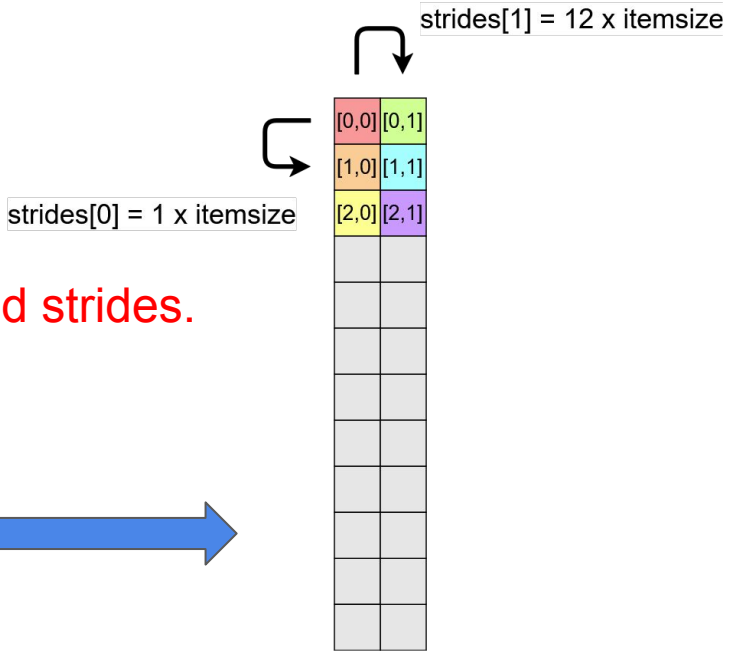
- `temps2d_T.shape` `(12, 2)`
- `temps2d_T.strides` `(4, 48)`



NumPy - Strides & Swapping axes

- `temps2d.shape` `(2, 12)`
- `temps2d.strides` `(48, 4)`





Swapping axes: Simply reordering the shape and strides.

```
temps2d_T = temps2d.T
```

- `temps2d_T.shape` `(12, 2)`
- `temps2d_T.strides` `(4, 48)`

NumPy - Operations which create views

- **Reshaping**
- **Swapping axes** (e.g., transpose)
- **Basic indexing**
- **Broadcasting**
- **…**

NumPy - Strides & Basic indexing

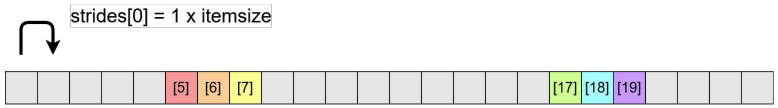
Extract the temperature of the summer months.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
```

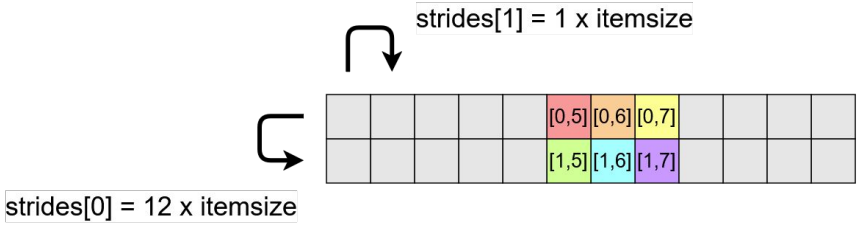
```
summer_months = temps2d[:, 5:8]

print(summer_months)                            # [[21.6 23.1 24.2]
                                                #  [25.1 28.3 24.3]]
print(summer_months.shape)                      # (2, 3)
print(summer_months.strides)                    # (48, 4)
print(summer_months.base is temps2d)            # False
print(summer_months.base is temperatures)       # True
```



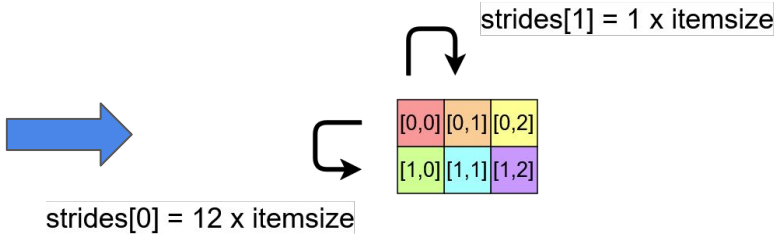
NumPy - Strides & Basic indexing

- `temps2d.shape` `(2, 12)`
- `temps2d.strides` `(48, 4)`




```
summer_months = temps2d[:, 5:8]
```

- `summer_months.shape` `(2, 3)`
- `summer_months.strides` `(48, 4)`



NumPy - Strides & Basic indexing

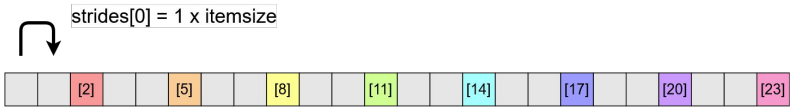
Extract the temperatures of the first months of every season.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
```

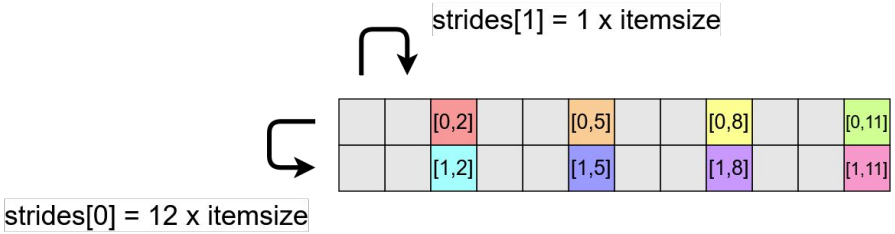
```
first_m_of_seasons = temps2d[:, 2::3]

print(first_m_of_seasons.shape)                    # (2, 4)
print(first_m_of_seasons.strides)                  # (48, 12)
print(first_m_of_seasons.base is temps2d)          # False
print(first_m_of_seasons.base is temperatures)     # True
```



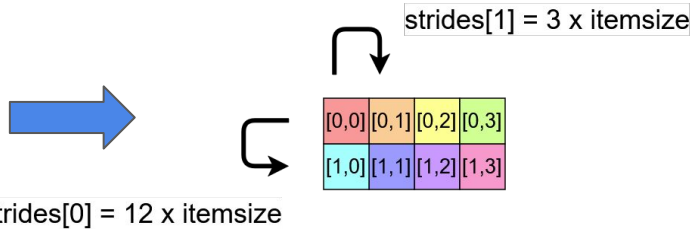
NumPy - Strides & Basic indexing

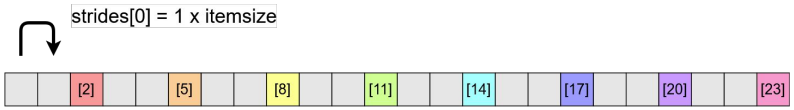
- `temps2d.shape` `(2, 12)`
- `temps2d.strides` `(48, 4)`




```
first_m_of_seasons = temps2d[:, 2::3]
```

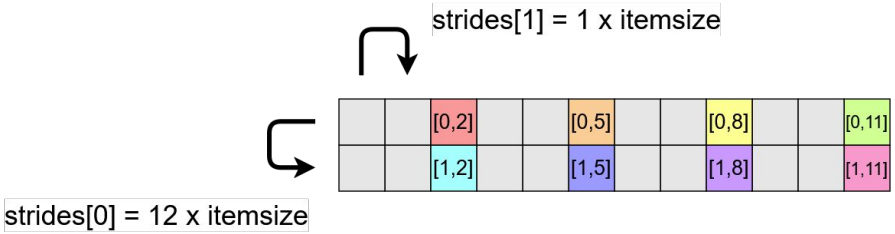
- `first_m_of_seasons.shape` `(2, 4)`
- `first_m_of_seasons.strides` `(48, 12)`





NumPy - Strides & Basic indexing

- `temps2d.shape` `(2, 12)`
- `temps2d.strides` `(48, 4)`

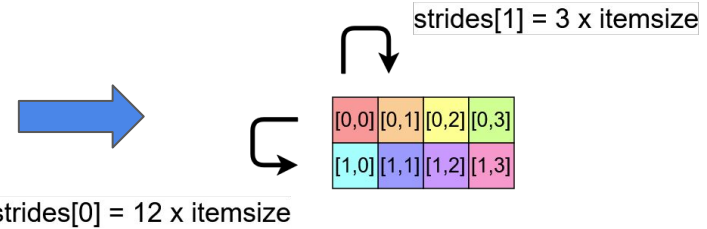


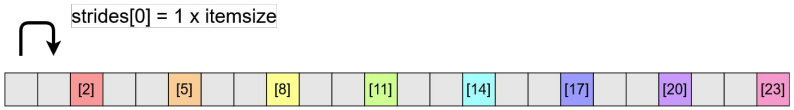
Basic indexing:

- **Beginning of slice:** Shifting the address of the first index if needed.
- **End of slice:** Shorter shape along axis if needed.
- **Step size:** Multiply stride along axis if not 1.

```
first_m_of_seasons = temps2d[:, 2::3]
```

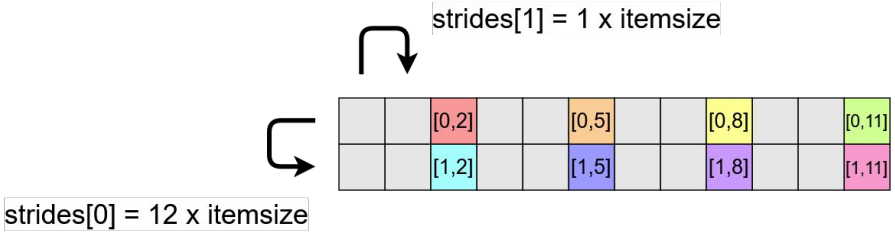
- `first_m_of_seasons.shape` `(2, 4)`
- `first_m_of_seasons.strides` `(48, 12)`





NumPy - Strides & Basic indexing

- `temps2d.shape` **(2, 12)**
- `temps2d.strides` **(48, 4)**

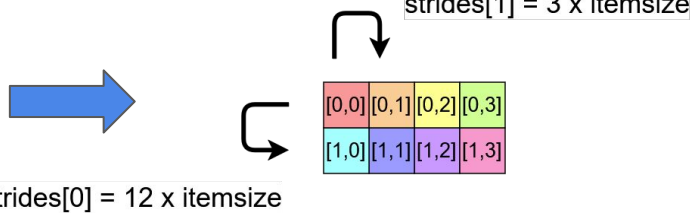


Basic indexing:

- **Beginning of slice:** Shifting the address of the first index if needed.
- **End of slice:** Shorter shape along axis if needed.
- **Step size:** Multiply stride along axis if not 1.
  Stride can be negative!
  (e.g. for `temps2d[:, ::-1]` → `(48, -4)`)

```
first_m_of_seasons = temps2d[:, 2::3]
```

- `first_m_of_seasons.shape` **(2, 4)**
- `first_m_of_seasons.strides` **(48, 12)**



NumPy - Strides & Basic indexing

Extract the temperature of the summer months into an 1D array.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
```

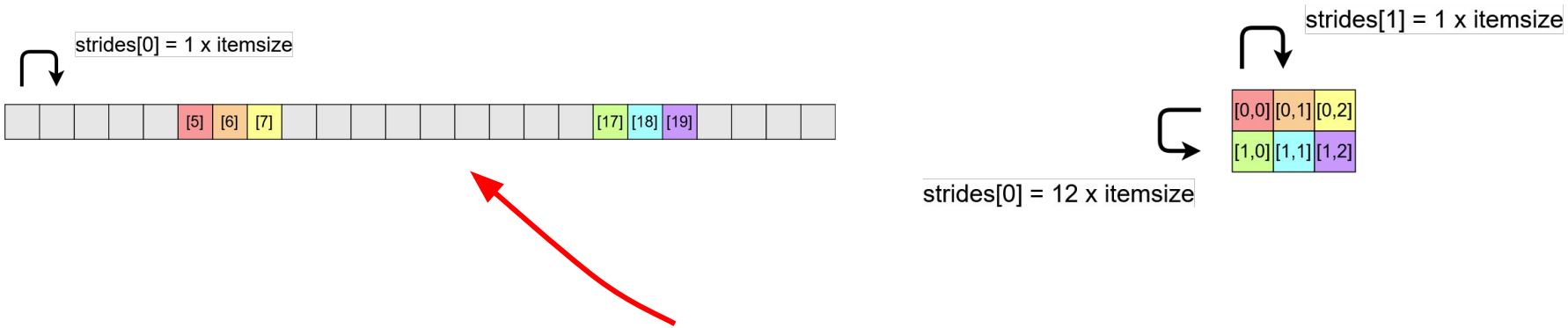
```
summer_months = temps2d[:, 5:8]

summer_months_1d = summer_months.reshape(-1)
print(summer_months_1d)                           # [21.6 23.1 24.2 25.1 28.3 24.3]
print(summer_months_1d.shape)                     # (6,)
print(summer_months_1d.strides)                   # (4,)
print(summer_months_1d.base is temperatures)      # False
```

Certain combinations of the view-producing operations will not produce a view.

NumPy - Strides & Basic indexing

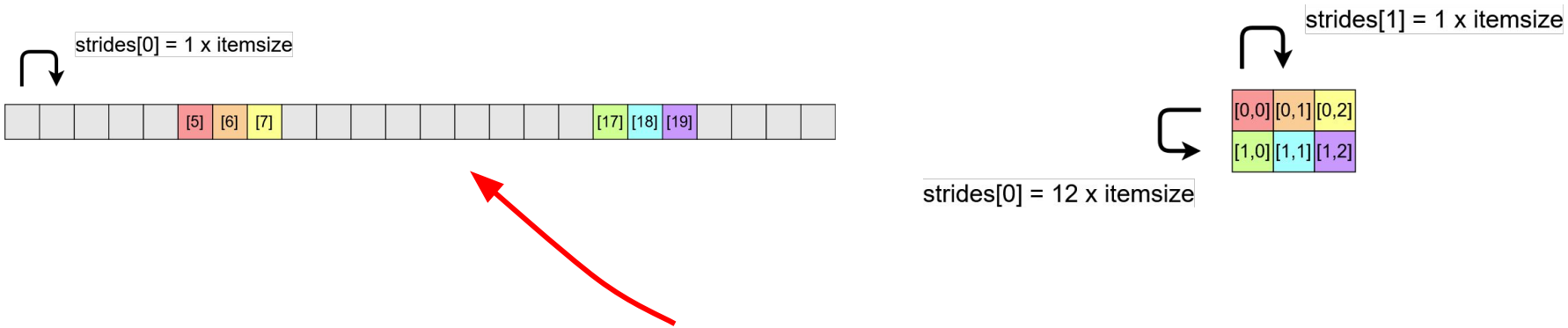
Certain combinations of the view-producing operations **will not produce a view.**



Example: Basic indexing, then reshaping

NumPy - Strides & Basic indexing

Certain combinations of the view-producing operations **will not produce a view.**



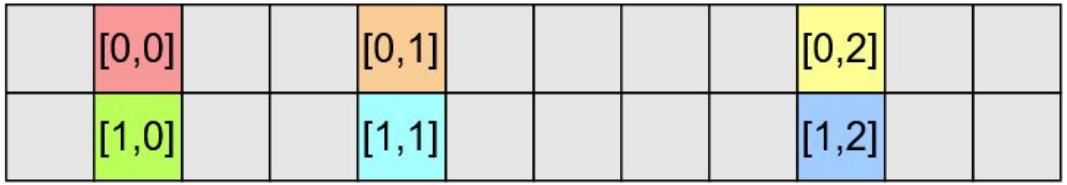
Example: Basic indexing, then reshaping

In 2D, a combination of two regular step sizes could describe the memory address of all elements, but when reshaped to 1D,
a single regular step size cannot describe all of these elements anymore.
→ **The referred elements needs to be copied into a new array.**

NumPy - Strides & Advanced indexing

Advanced indexing will never produce a view.

temps2d[:, [1, 4, 9]]



In advanced indexing, arbitrary indices can be referred.
Therefore, regular strides (step sizes) usually cannot describe the memory address of all elements.

→ The referred elements needs to be copied into a new array.

NumPy - Operations which create views

- **Reshaping**
- **Swapping axes** (e.g., transpose)
- **Basic indexing**
- **Broadcasting**
- …

NumPy - Strides & Broadcasting

For each month, get the difference in temperature from the average of the corresponding year.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
```

```
yearly_avg_temps = np.mean(temps2d, axis=1)[:, None]    # (2, 1)
diffs = temps2d - yearly_avg_temps                      # (2, 12) v (2, 1)

print(diffs.shape)          # (2, 12)
print(diffs)                # [[-11.383332   -12.383332    -4.983332     ...  -14.483332  ]
                            #  [-15.250001   -14.450001    -7.850001     ...  -15.35      ]]
```

NumPy - Strides & Broadcasting

For each month, get the difference in temperature from the average of the corresponding year.

```
temps2d = temperatures.reshape(2, 12)     # [[-0.5 -1.5  5.9 12.9 18.1 21.6 23.1 24.2 15.  11.7  3.7 -3.6]
                                          #  [-4.3 -3.5  3.1 13.5 17.4 25.1 28.3 24.3 18.5 13.1  0.3 -4.4]]

print(temps2d.shape)                      # (2, 12)
print(temps2d.strides)                    # (48, 4)
```

```
yearly_avg_temps = np.mean(temps2d, axis=1)[:, None]    # (2, 1)
diffs = temps2d - yearly_avg_temps                      # (2, 12) v (2, 1)

print(diffs.shape)          # (2, 12)
print(diffs)                # [[-11.383332  -12.383332   -4.983332   ...  -14.483332  ]
                            #  [-15.250001  -14.450001   -7.850001   ...  -15.35      ]]
```

Problem: Broadcasting is automatic here, only a temporary view is created which we cannot observe.

NumPy - Explicit broadcasting

`np.broadcast_to`:
Perform broadcasting of an array to a given shape. A view is created.

Explicit broadcasting

```
yearly_avg_temps = np.mean(temps2d, axis=1)        # (2,)
yearly_avg_temps_2d = yearly_avg_temps[:, None]    # (2, 1)

yearly_avg_temps_broadcasted = np.broadcast_to(yearly_avg_temps_2d, temps2d.shape)     # (2, 1) -> (2, 12)
print(yearly_avg_temps_broadcasted.shape)                                              # (2, 12)
print(yearly_avg_temps_broadcasted.strides)                                            # (4, 0)
print(yearly_avg_temps_broadcasted.base is yearly_avg_temps)                           # True
print(yearly_avg_temps_broadcasted.base is yearly_avg_temps_2d)                        # False

print(yearly_avg_temps_broadcasted)                # [[10.883332 10.883332 10.883332 10.883332 ... 10.883332]
                                                   #  [10.950001 10.950001 10.950001 10.950001 ... 10.950001]]

diffs = temps2d - yearly_avg_temps_broadcasted     # (2, 12) v (2, 12)
```

NumPy - Strides & Broadcasting

For each month, get the difference in temperature from the average of the corresponding year.

```
yearly_avg_temps = np.mean(temps2d, axis=1)        # (2,)
yearly_avg_temps_2d = yearly_avg_temps[:, None]    # (2, 1)

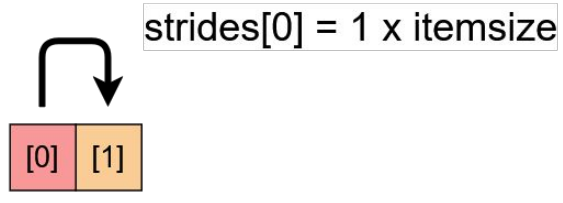
yearly_avg_temps_broadcasted = np.broadcast_to(yearly_avg_temps_2d, temps2d.shape)    # (2, 1) -> (2, 12)
print(yearly_avg_temps_broadcasted.shape)                                             # (2, 12)
print(yearly_avg_temps_broadcasted.strides)                                           # (4, 0)
print(yearly_avg_temps_broadcasted.base is yearly_avg_temps)                          # True
print(yearly_avg_temps_broadcasted.base is yearly_avg_temps_2d)                       # False

print(yearly_avg_temps_broadcasted)                # [[10.883332 10.883332 10.883332 10.883332 ... 10.883332]
                                                   #  [10.950001 10.950001 10.950001 10.950001 ... 10.950001]]

diffs = temps2d - yearly_avg_temps_broadcasted     # (2, 12) v (2, 12)
```

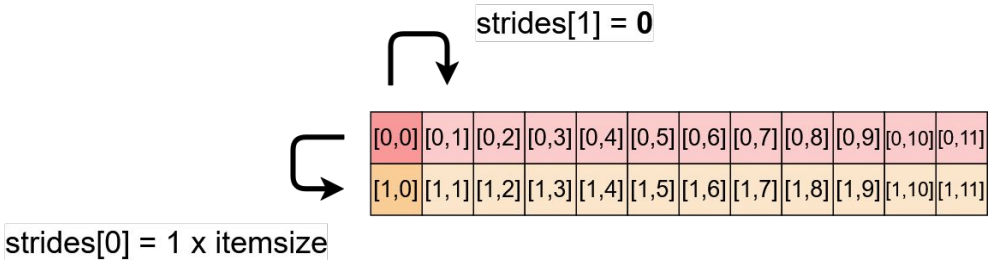
NumPy - Strides & Broadcasting

- `yearly_avg_temps.shape` `(2,)`
- `yearly_avg_temps.itemsize` `4`
- `yearly_avg_temps.strides` `(4,)`



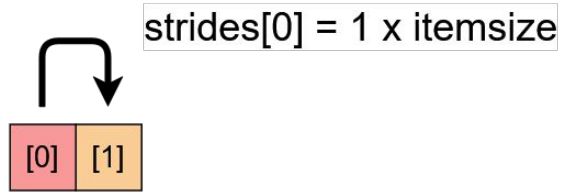

```
yearly_avg_temps_2d = yearly_avg_temps[:, None]
yearly_avg_temps_broadcasted = np.broadcast_to(yearly_avg_temps_2d, shape=(2, 12))
```

- `yearly_avg_temps_broadcasted.shape` `(2, 12)`
- `yearly_avg_temps_broadcasted.strides` `(4, 0)`



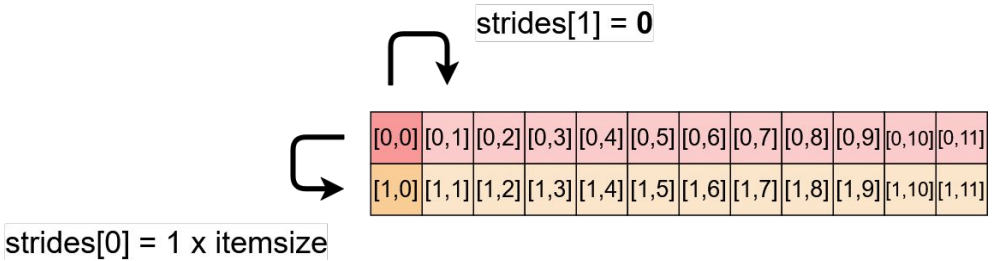
NumPy - Strides & Broadcasting

- `yearly_avg_temps.shape` `(2,)`
- `yearly_avg_temps.itemsize` `4`
- `yearly_avg_temps.strides` `(4,)`



Broadcasting: Virtually repeated axes have a stride of 0.
Accessing different indices along this axis **will point to the same memory address.**

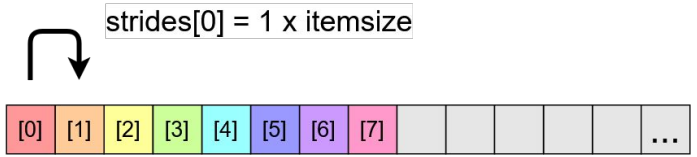
- `yearly_avg_temps_broadcasted.shape` `(2, 12)`
- `yearly_avg_temps_broadcasted.strides` `(4, 0)`



NumPy - Operations which create views

- **Reshaping**
- **Swapping axes** (e.g., transpose)
- **Basic indexing**
- **Broadcasting**
- **`np.lib.stride_tricks.as_strided()`**
- **…**

NumPy - Stride tricks

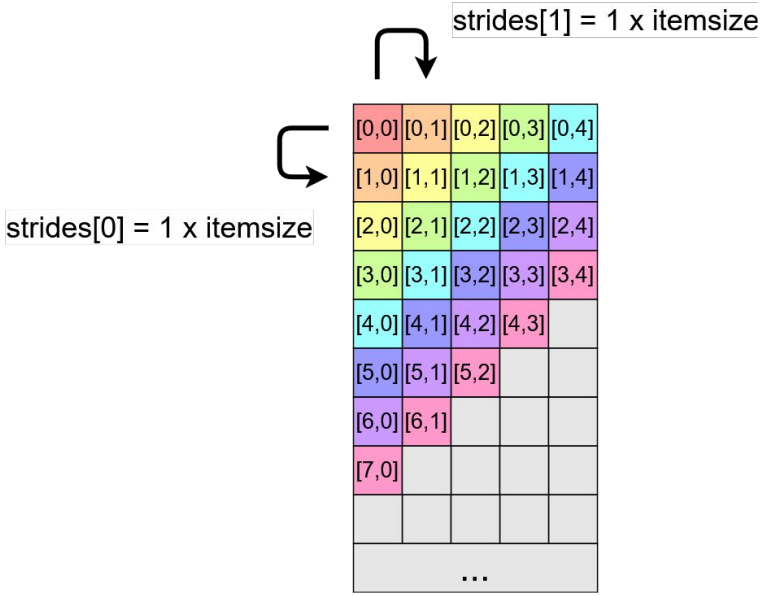


Explicit specification of the shape and strides of a new view.

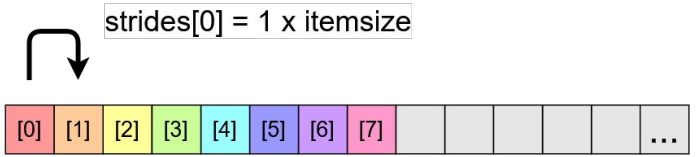
```
n_bytes_per_elem = temperatures.itemsize      # 4
windowed_temps = np.lib.stride_tricks.as_strided(
                    temperatures,
                    shape=(20, 5),
                    strides=(n_bytes_per_elem, n_bytes_per_elem),
                    writeable=False)

print(windowed_temps.shape)                          # (20, 5)
print(windowed_temps.strides)                        # (4, 4)

print(np.mean(windowed_temps, axis=1).shape)         # (20,)
```



NumPy - Stride tricks

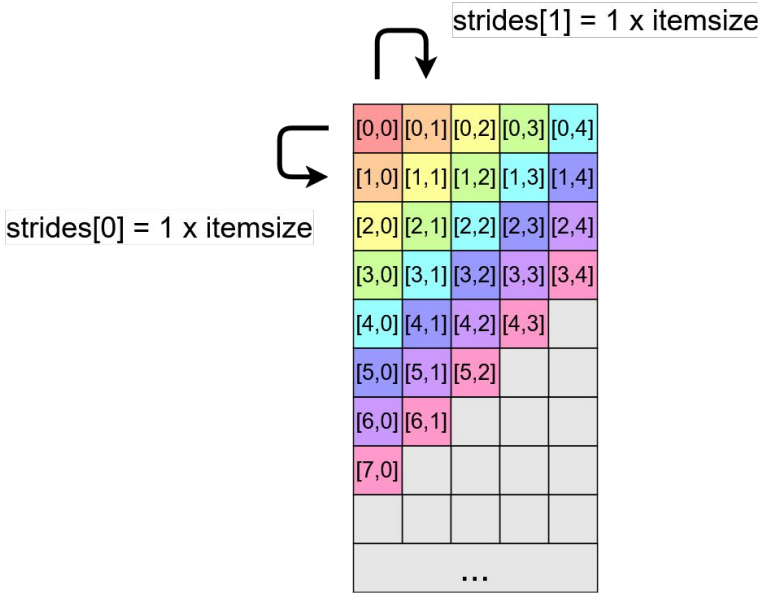


Explicit specification of the shape and strides of a new view.

```
n_bytes_per_elem = temperatures.itemsize      # 4
windowed_temps = np.lib.stride_tricks.as_strided(
                    temperatures,
                    shape=(20, 5),
                    strides=(n_bytes_per_elem, n_bytes_per_elem),
                    writeable=False)

print(windowed_temps.shape)                          # (20, 5)
print(windowed_temps.strides)                        # (4, 4)

print(np.mean(windowed_temps, axis=1).shape)         # (20,)
```



It is not forbidden to have multiple indices pointing to the same memory location…
Example: Operations over the sliding window views of a time series.

NumPy - Operations which create views

- **Reshaping**
- **Swapping axes** (e.g., transpose)
- **Basic indexing**
- **Broadcasting**
- **`np.lib.stride_tricks.as_strided()`**
- **`...`**